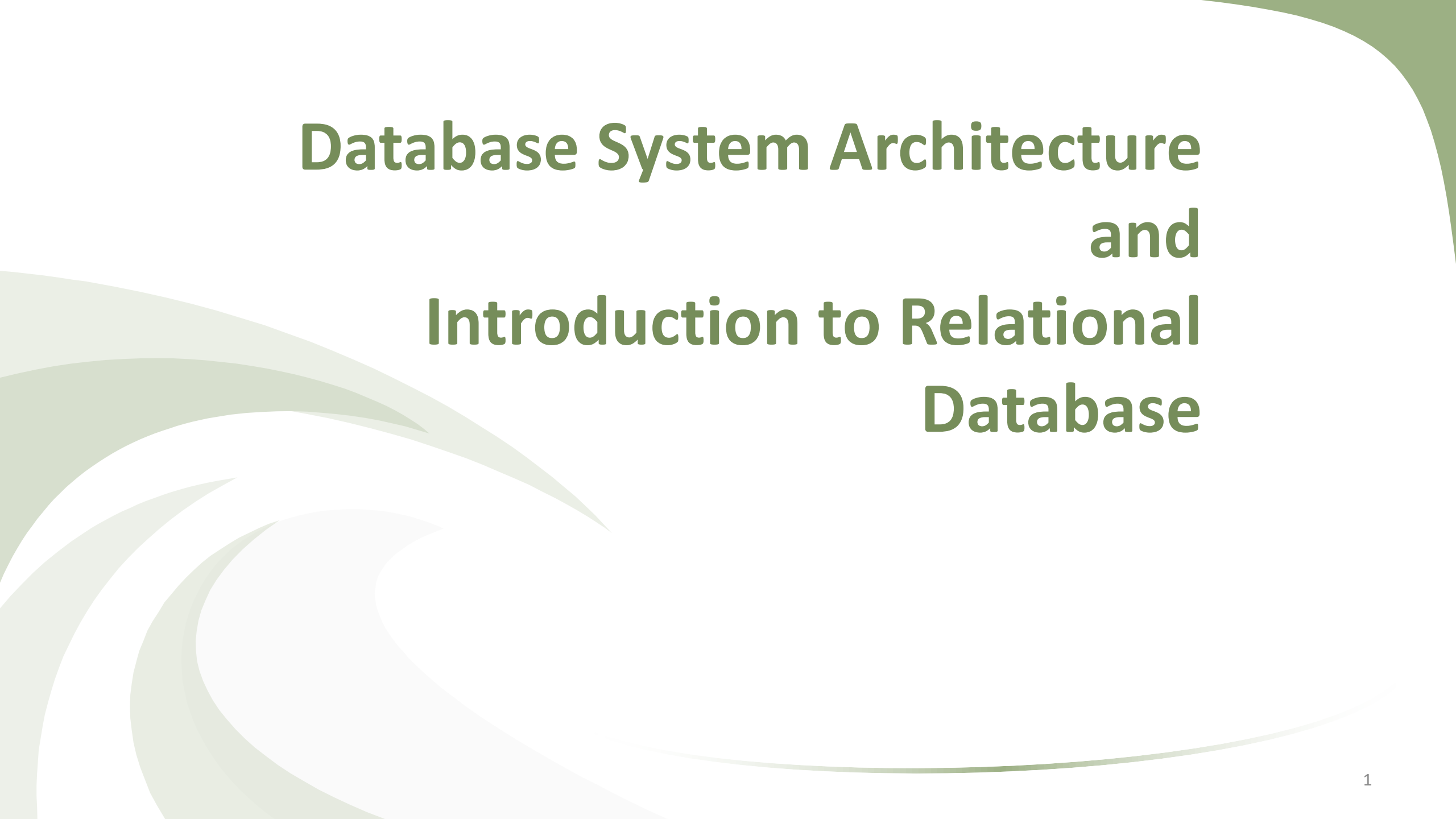


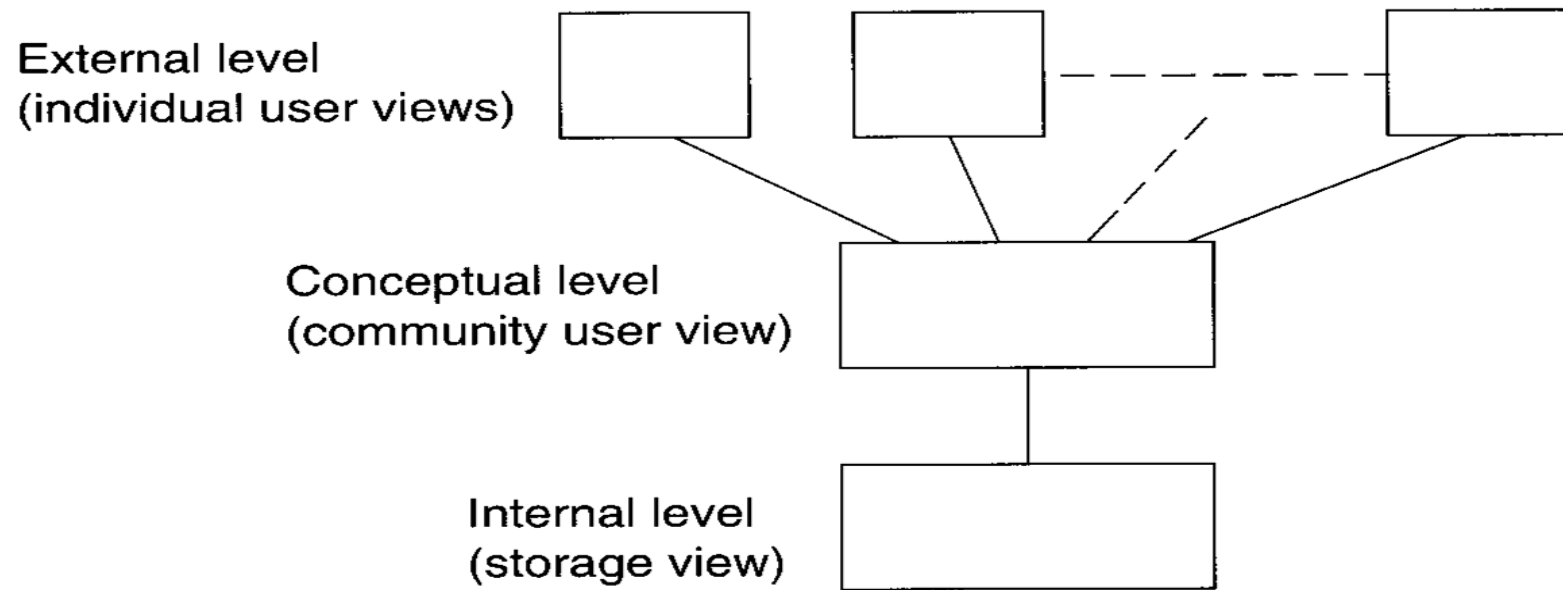


# **Database System Architecture and Introduction to Relational Database**

# Database System Architecture

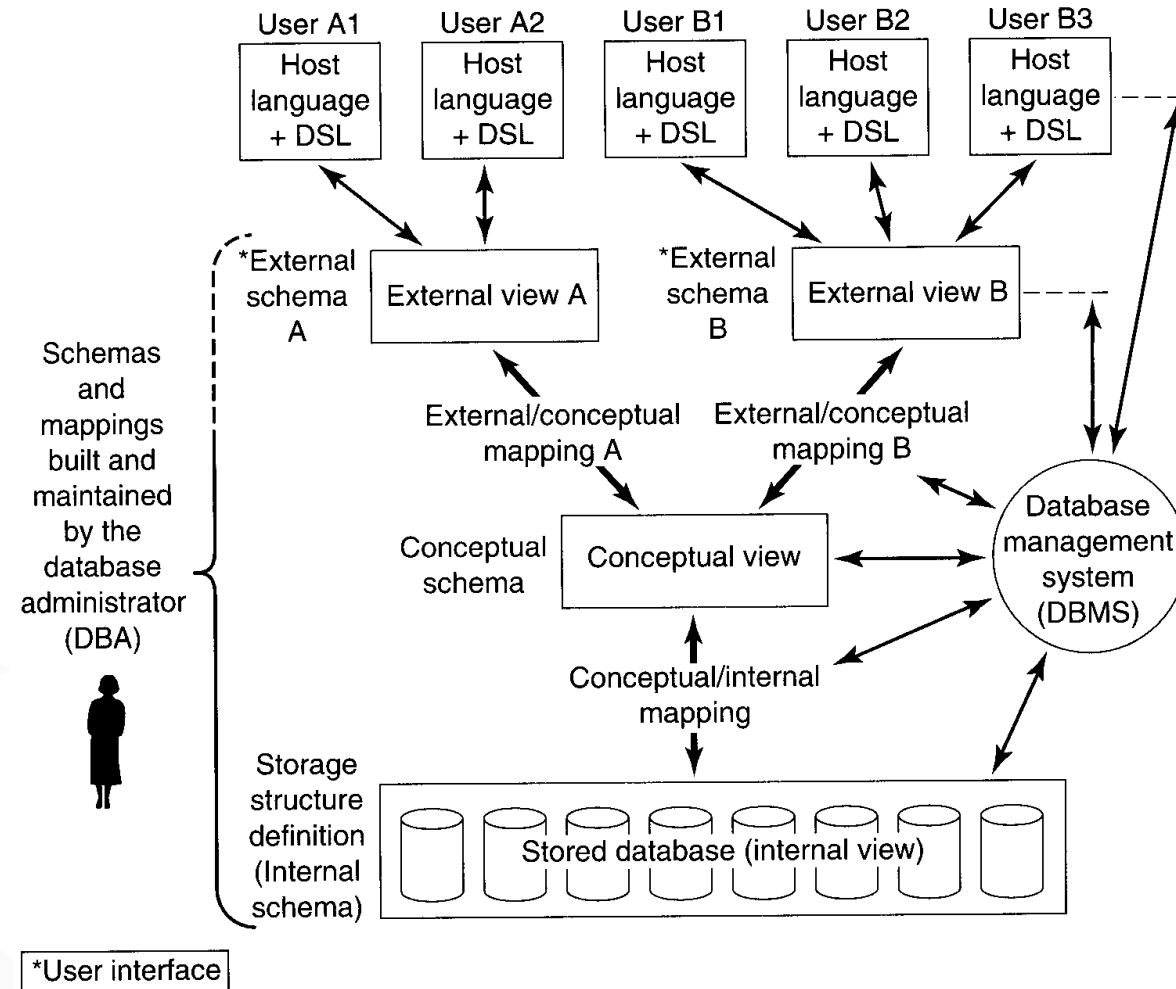
- ANSI/SPARC Architecture is divided into three level known as :
  - Internal level (physical level) closest to physical storage
  - External level (user logical level) closest to the users
  - Conceptual level (logical level) between the other two

# Database System Architecture (con't)



The three levels of the architecture

# Database System Architecture (con't)



Detailed system architecture

# Data and Its Structure

- Data is actually stored as bits, but it is difficult to work with data at this level.
- It is convenient to view data at different levels of abstraction.
- **Schema:** Description of data at some level. Each level has its own schema.
- We will be concerned with three schemas: physical, conceptual, and external.

# Physical Data Level

- Physical schema describes details of how data is stored: tracks, cylinders, indices etc.
- Early applications worked at this level - explicitly dealt with details.
- **Problem:** Routines hardcoded to deal with physical representation.
  - Changes to data structure difficult to make.
  - Application code becomes complex since it must deal with details.
  - Rapid implementation of new features impossible.

# Conceptual Data Level

- Hides details.
  - In the relational model, the conceptual schema presents data as a set of tables.
- DBMS maps from conceptual to physical schema automatically.
- Physical schema can be changed without changing application:
  - DBMS must change mapping from conceptual to physical.
- Referred to as *physical data independence*.

# Conceptual Data Level (con't)

Application

Conceptual  
view of data

DBMS

Physical  
view of data



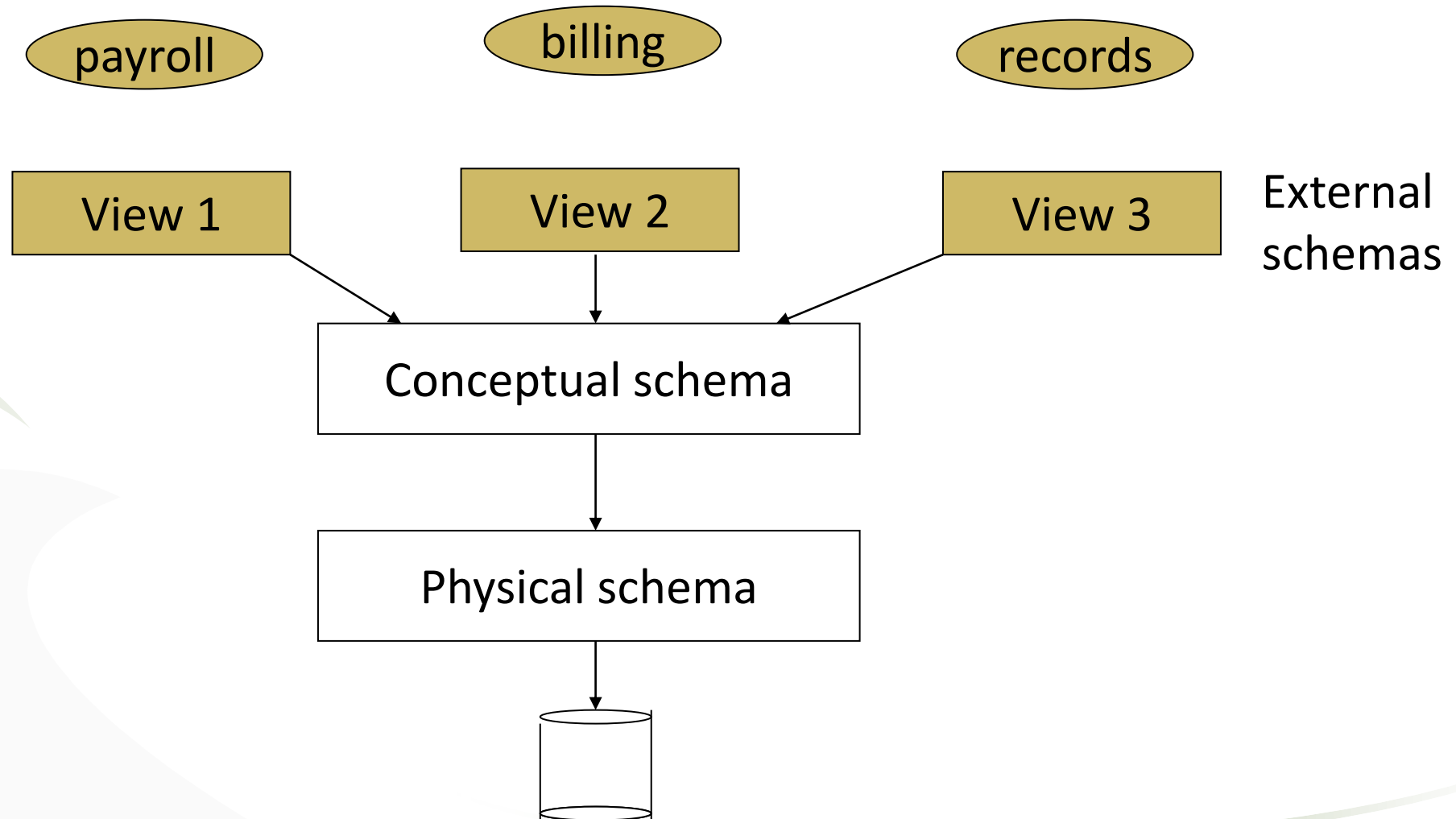
# External Data Level

- In the relational model, the external schema also presents data as a set of relations.
- An external schema specifies a *view* of the data in terms of the conceptual level. It is tailored to the needs of a particular category of users.
  - Portions of stored data should not be seen by some users.
    - Students should not see faculty salaries.
    - Faculty should not see billing data.
  - Information that can be derived from stored data might be viewed as if it were stored.
    - GPA not stored, calculated when needed.

## External Data Level (con't)

- Application is written in terms of an external schema.
- A view is computed when accessed (not stored).
- Different external schemas can be provided to different categories of users.
- Translation from external to conceptual done automatically by DBMS at run time.
- Conceptual schema can be changed without changing application:
  - Mapping from external to conceptual must be changed.
- Referred to as *conceptual data independence*.

# Levels of Abstraction



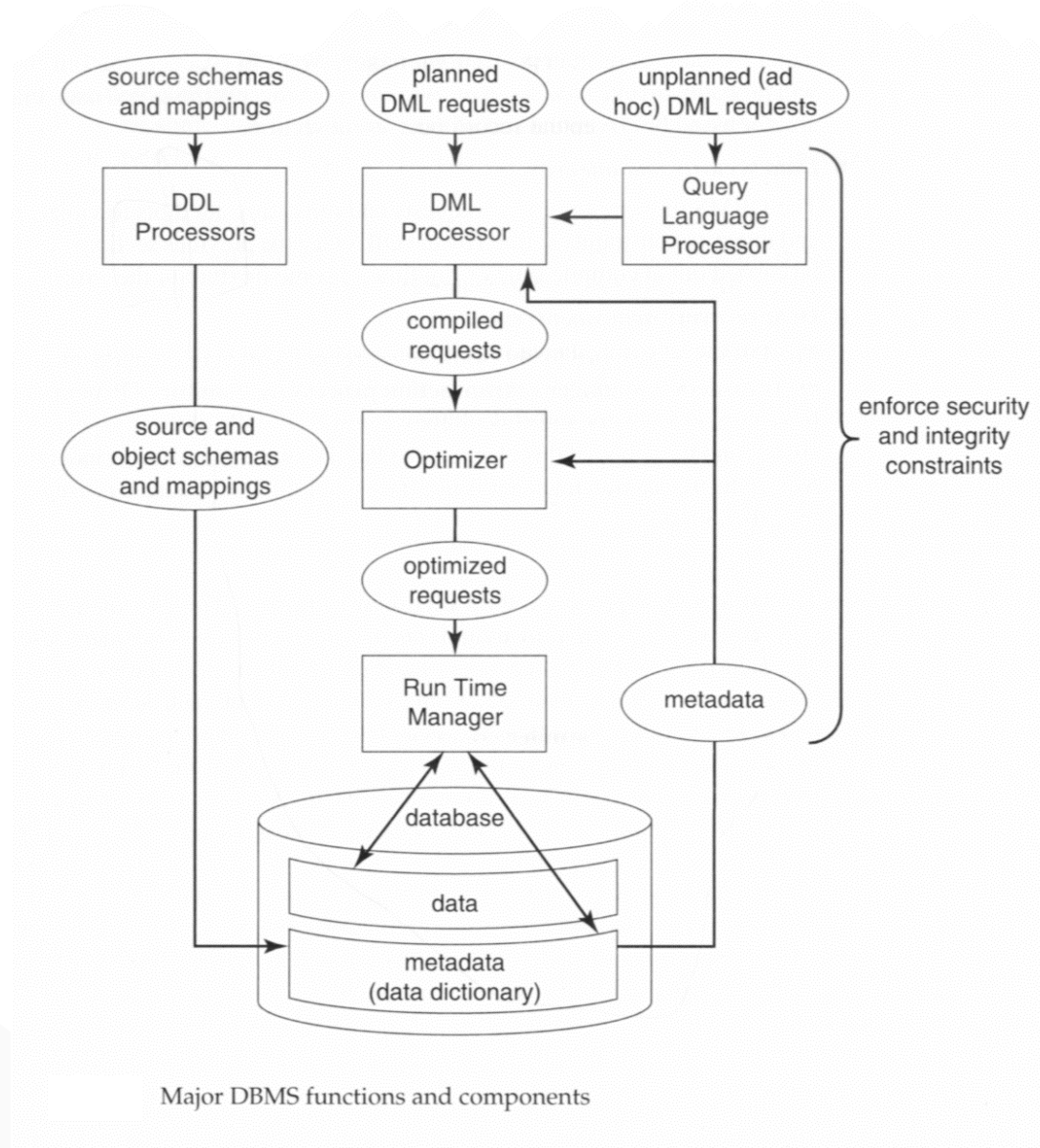
# The Database Administrator

- Defining the conceptual schema
- Defining the internal schema
- Liaising with users
- Defining security and integrity constraints
- Defining dump and reload policies
- Monitoring performance and responding to changing requirements

# DBMS Functions

- Data definition (DDL)
- Data manipulation (DML)
- Optimization and execution
- Data security and integrity
- Data recovery and concurrency
- Data dictionary
- Performance

# DBMS Functions (con't)



# Data Model

- **Schema:** description of data at some level (*e.g.*, tables, attributes, constraints, domains)
- **Model:** tools and language for describing:
  - Conceptual and external schema
    - Data definition language (DDL)
  - Integrity constraints, domains (DDL)
  - Operations on data
    - Data manipulation language (DML)
  - Directives that influence the physical schema (affects performance, not semantics)
    - Storage definition language (SDL)

# Relational Model

- A particular way of structuring data (relations)
- Simple
- Mathematically based
  - Expressions (queries) can be analyzed by DBMS
  - Transformed to equivalent expressions automatically (query optimization)
    - Optimizers have limits ( $\Rightarrow$  programmer needs to know how queries are evaluated and optimized)



# Relation Instance

- Relation is a set of tuples
  - Tuple ordering immaterial
  - No duplicates
  - Cardinality of relation = number of tuples
- All tuples in a relation have the same structure; constructed from the same set of attributes
  - Attributes named ( $\Rightarrow$  ordering immaterial)
  - Value of an attribute drawn from the attribute's domain
  - Degree or Arity = number of attributes

# A Relation

AUTHOR(Author\_ID: char, lastname: varchar, firstname: varchar, email: varchar, city: varchar, country: char)

## Relation Schema

## Relation Instance

**DEGREE=6**  
**CARDINALITY=5**

**TUPLES**

**ATTRIBUTES**

| Author_ID | Lastname | Firstname | Email       | City         | Country |
|-----------|----------|-----------|-------------|--------------|---------|
| A1        | Chong    | Raul      | rfc@ibm.com | Toronto      | CA      |
| A2        | Ahuja    | Rav       | ra@ibm.com  | Toronto      | CA      |
| A3        | Hakes    | Ian       | ih@ibm.com  | Toronto      | CA      |
| A4        | Sharma   | Neeraj    | ns@ibm.com  | Chennai      | IN      |
| A5        | Perniu   | Liviu     | lp@univ.com | Transilvania | RO      |

A **DOMAIN** is the set of all possible values for a specific attribute

# Relation Instance (Example)

| Id      | Name | Address    | Status    |
|---------|------|------------|-----------|
| 1111111 | John | 123 main   | freshman  |
| 2345678 | Mary | 456 cedar  | sophomore |
| 4433322 | Art  | 77 so. 3rd | senior    |
| 7654321 | Pat  | 88 no. 4th | sophomore |

Student

# Relation Schema

- Relation name
- Attribute names and domains
- Integrity constraints - e.g.,:
  - The values of a particular attribute in all tuples are unique
  - The values of a particular attribute in all tuples are greater than 0
- Default values

# Relational Database

- Finite set of relations
- Each relation consists of a schema and an instance
- Database schema = set of relation schemas (and other things)
- Database instance = set of (corresponding) relation instances

# Database Schema (Example)

- Student (Id: INT, Name: STRING, Address: STRING, Status: STRING)
- Professor (Id: INT, Name: STRING, DeptId: DEPTS)
- Course (DeptId: DEPTS, CrsName: STRING, CrsCode: COURSES)
- Transcript (CrsCode: COURSES, StudId: INT, Grade: GRADES, Semester: SEMESTERS)
- Department(DeptId: DEPTS, Name: STRING)

# Integrity Constraints

- Part of schema
- Restriction on state (or sequence of states) of data base
- Enforced by DBMS
- Intra-relational - involve only one relation
  - Part of relation schema
  - e.g., all Ids are unique
- Inter-relational - involve several relations
  - Part of relation schema or database schema

# Kinds of Integrity Constraints

- Static - limitation on state of database
  - Syntactic (structural)
    - e.g., all values in a column must be unique
  - Semantic (involve meaning of attributes)
    - e.g., cannot register for more than 18 credits
- Dynamic - limitation on sequence of database states (supported by some DBMSs, but not in current SQL standard)
  - e.g., cannot raise salary by more than 5%



# Key Constraint

- Values in a column (or columns) of a relation are unique: at most one row in a relation instance can contain a particular value(s)
- **Key** - set of attributes satisfying key constraint
  - e.g., Id in Student,
  - e.g., (StudId, CrsCode, Semester) in Transcript

# Key Constraint (con't)

- Minimality - no subset of a key is a key
  - (StudId, CrsCode) is not a key of Transcript
- **Superkey** - set of attributes containing key
  - (Id, Name) is a superkey of Student
- Every relation has a key
- Relation can have several keys:
  - **primary key** (Id in Student) – (cannot be null)
  - **candidate key** ((Name, Address) in Student)

# Keys

- There are many type of keys in relational model
  - Candidate keys
  - Primary keys
  - Alternate keys
  - Foreign keys

# Candidate Keys

- The set of all attribute of relation has a the uniqueness property
- No two tuples in the value of relation are duplicates of one another
- Let  $K$  be a set of attributes of relation  $R$ . Then  $K$  is a candidate key for  $R$  iff it processes both of the 'uniqueness' and 'irreducibility' properties

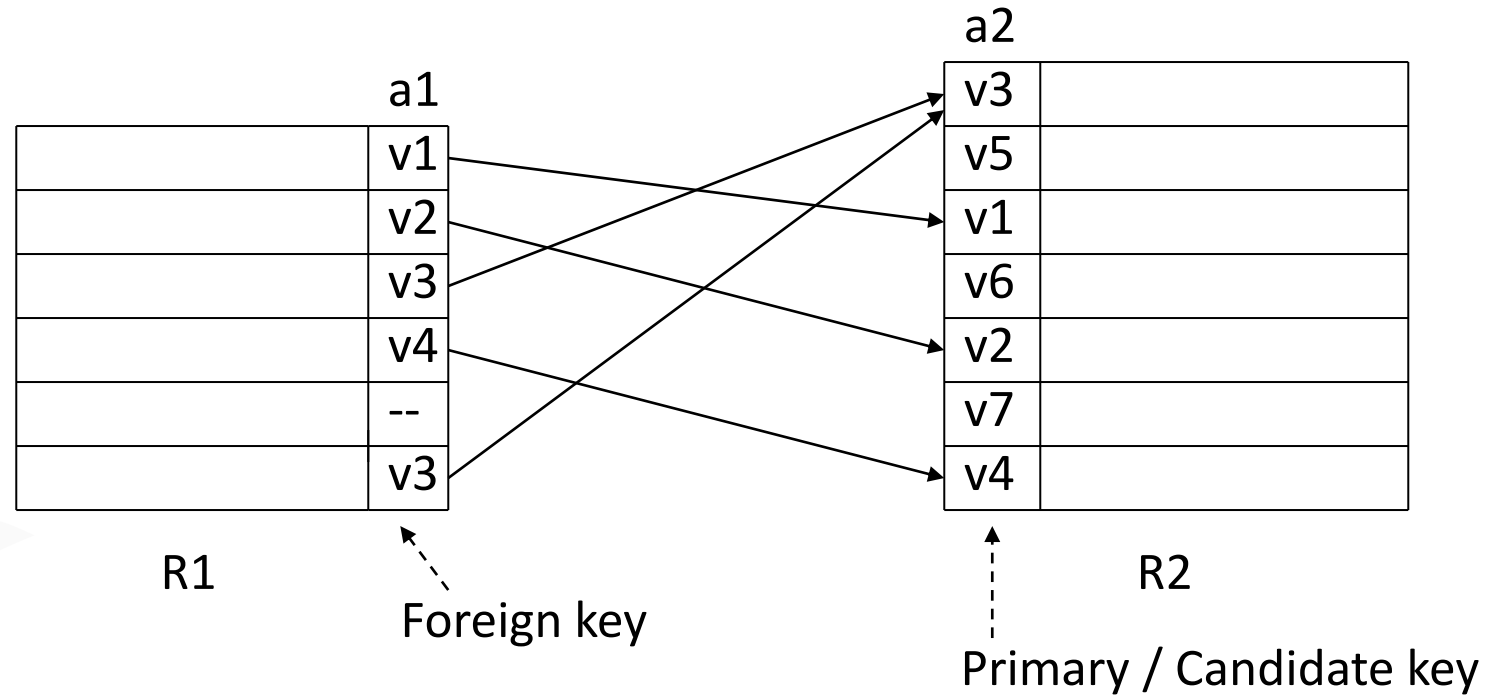
# Primary Keys, Alternate Keys and Foreign Keys

- One of candidate keys be chosen as the primary key
- The others are then called alternate keys
- A foreign key is a set of attributes of one relation whose values are required to matches of some candidate key of some relation

# Foreign Key Constraint

- **Referential integrity** => Item named in one relation must correspond to tuple(s) in another that describes the item
  - Transcript (CrsCode) references Course(CrsCode )
  - Professor(DeptId) references Department(DeptId)
- a1 is a **foreign key** of R1 referring to a2 in R2 => if v is a value of a1, there is a *unique* tuple of R2 in which a2 has value v
  - This is a special case of referential integrity: a2 must be a candidate key of R2 (CrsCode is a key of Course)
  - If no row exists in R2 => violation of referential integrity
  - Not all rows of R2 need to be referenced.: relationship is not symmetric (some course might not be taught)
  - Value of a foreign key might not be specified (DeptId column of some professor might be null)

# Foreign Key Constraint (Example)



# Foreign Key (con't)

- Names of a1 and a2 need not be the same.
  - With tables:  
Teaching(CrsCode: COURSES, Sem: SEMESTERS, ProfId: INT)  
Professor(Id: INT, Name: STRING, DeptId: DEPTS)  
ProfId attribute of Teaching references Id attribute of Professor
- R1 and R2 need not be distinct.
  - Employee(Id:INT, MgrId:INT, ....)
    - Employee(MgrId) references Employee(Id)
  - Every manager is also an employee and hence has a unique row in Employee



# Foreign Key (con't)

- Foreign key might consist of several columns
  - (CrsCode, Semester) of Transcript references (CrsCode, Sem) of Teaching
- $R1(a_1, \dots, a_n)$  references  $R2(b_1, \dots, b_n)$ 
  - There exists a 1 - 1 relationship between  $a_1, \dots, a_n$  and  $b_1, \dots, b_n$
  - $a_i$  and  $b_i$  have same domains (although not necessarily the same names)
  - $b_1, \dots, b_n$  is a candidate key of  $R2$

# Inclusion Dependency

- Referential integrity constraint that is not a foreign key constraint
- Teaching(CrsCode, Semester) references Transcript(CrsCode, Semester) (no empty classes)
  - Reverse relationship *is* a foreign key
- Target attributes are not a candidate key
- No simple enforcement mechanism in SQL

# Semantic Constraints

- Domain, primary key, and foreign key are examples of structural (syntactic) constraints
- **Semantic constraints** express rules of application:
  - e.g., number of registered students  $\leq$  maximum enrollment